TRABALHO DE ORDENAÇÃO

Disciplina SIN 213 - Projeto de Algoritmos

📌 **Data de Entrega:** 27 de outubro
📌 **Valor:** 20 vistos, distribuídos da seguinte forma:

- **10 vistos** – Relatório (entrega obrigatória via Moodle)
- **10 vistos** – Código (no Drive, organizado na sua pasta correspondente)

ENTREVISTA DIA 27 NO HORÁRIO DA AULA

1. Entrega:

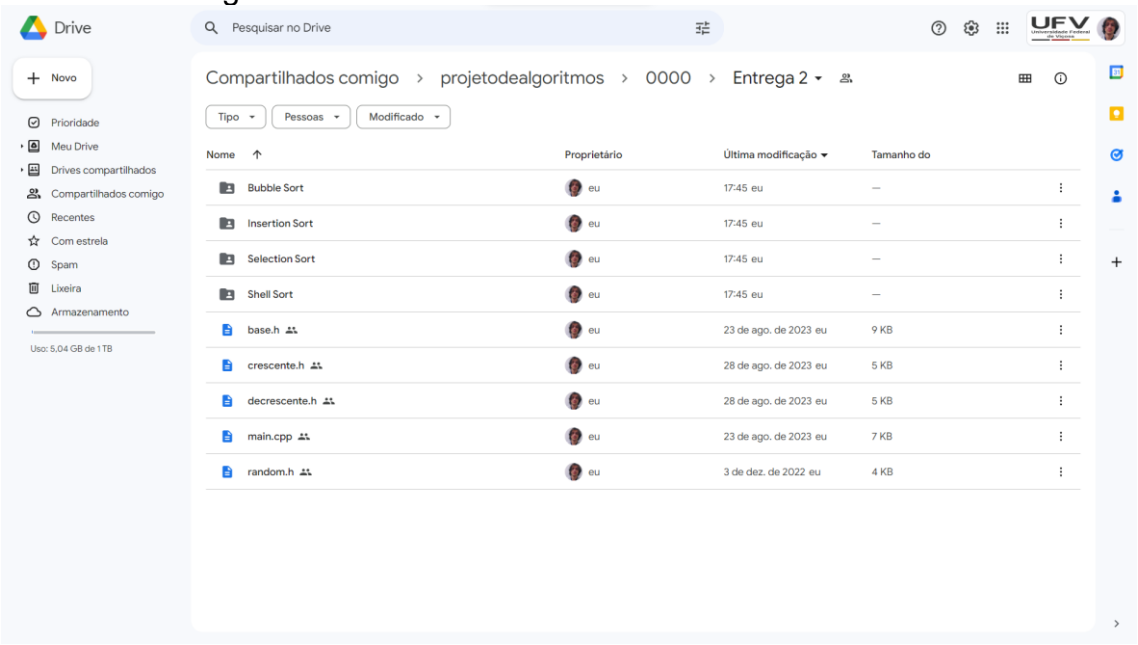
1.1. A entrega dos códigos (Código fonte e arquivos de entrada, tempo e o de saída, para cada instância do trabalho testando cada algoritmo deste trabalho) deverão ser colocadas em suas respectivas pastas do Drive, diferenciando por pastas de cada entrega.

1.2. A entrega do relatório deverá ser enviada no Moodle em formato PDF.

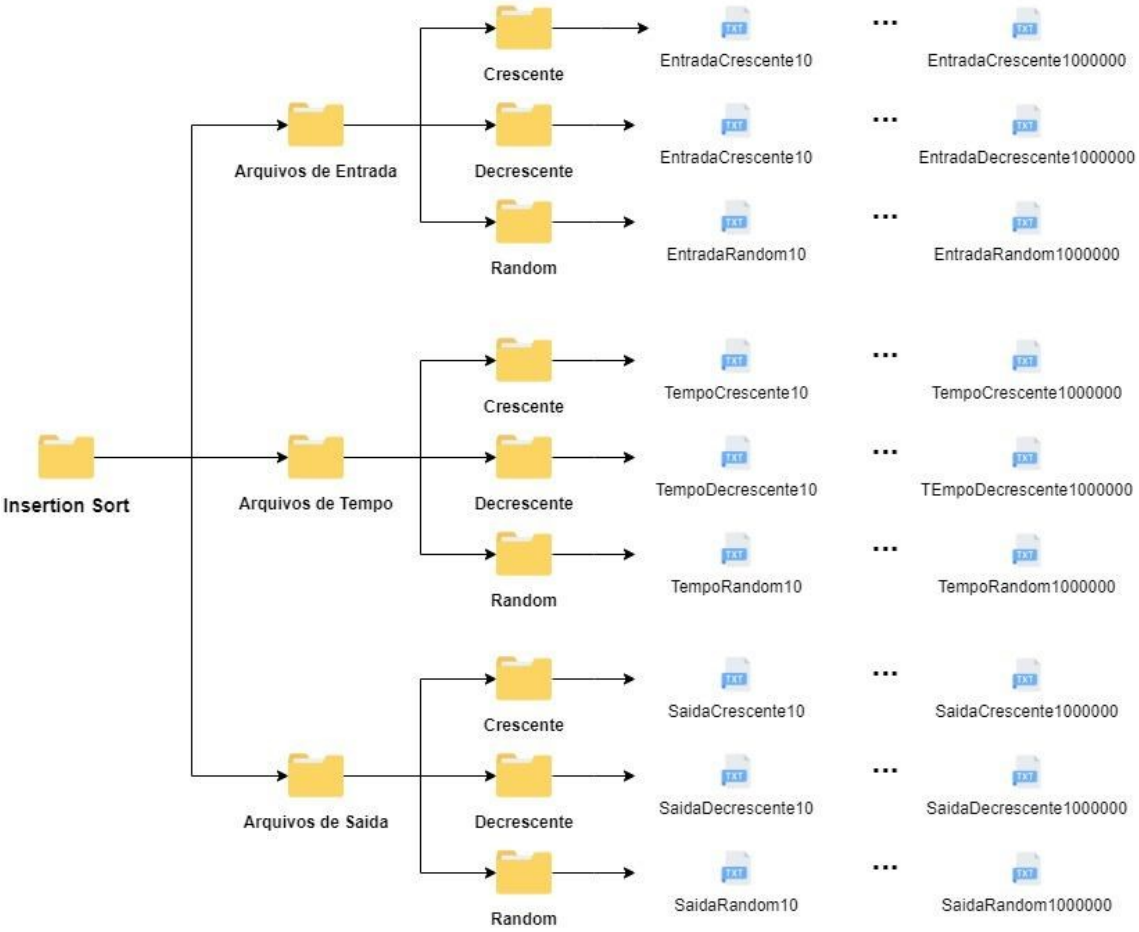
2. Exemplo de Organização:

2.1. Crie pastas para diferenciar cada entrega do seu trabalho.

2.2 Dentro da pasta deve ser enviado o seu código fonte e as pastas dos respectivos Algoritmos feitos, contendo os arquivos de entrada, saída e tempo conforme a imagem abaixo.



2.3. Cada pasta deve seguir o seguinte formato: **NÃO ESQUEÇA QUE DEVE SER FEITO A CRIAÇÃO DAS PASTAS E ARQUIVOS PELO CÓDIGO.**



Faça a criação do mesmo para o **insertion, selection, bubble e shellsort,** para poder fazer o relatório. **NÃO ESQUEÇA QUE DEVE SER FEITO A CRIAÇÃO DAS PASTAS E ARQUIVOS PELO CÓDIGO.**

Sugestão: Gere as entradas em um vetor e salve elas no arquivo de entrada com seu respectivo tamanho. Em seguida faça a ordenação do vetor, calculando o tempo gasto e salvando ela em um arquivo de tempo com seu respectivo tamanho. Por último salve o vetor organizado no arquivo de Saída com o seu respectivo tamanho.

3. Ordenação: Merge sort:

Ordenação é uma atividade necessária a vários sistemas de informação. Há vários algoritmos de ordenação, cada um deles com pontos fortes e pontos fracos. Os algoritmos **bolha**, **Inserção** e **Seleção** tem complexidade **O(N^2)** com relação a número de comparações de chave, mas dependendo dos dados, ambos podem ser **mais rápidos do que algoritmos com complexidade menor.**

3.1. Objetivo:

3.1.1. Experimentar algoritmos.

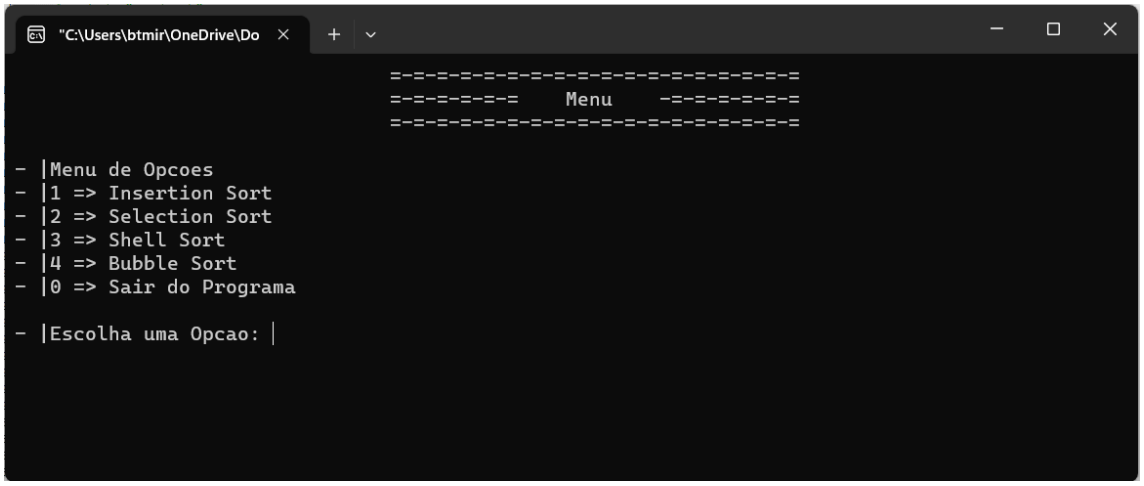
3.1.2. Comparar os tempos dos experimentos.

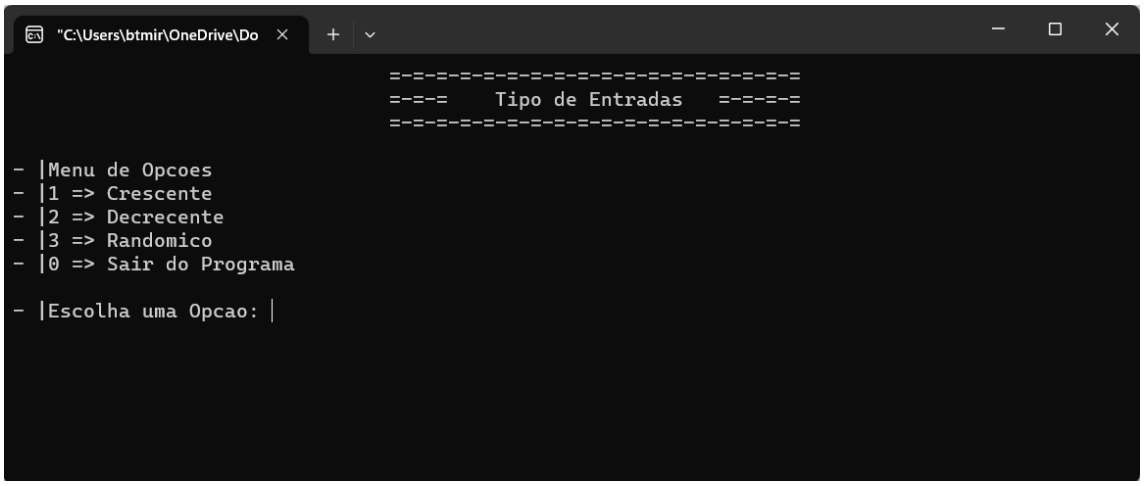
3.1.3. Analisar os tempos e especular as circunstâncias em que um algoritmo se sobressai ao outro.

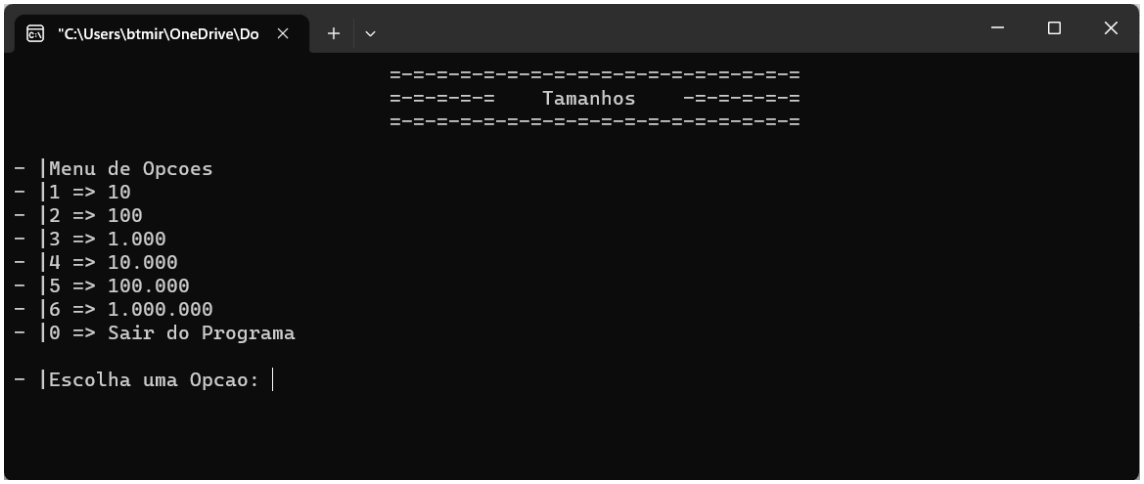
3.2. Código:

Implemente em **linguagem C ou C++** os algoritmos de ordenação **Insertion**, **Selection** , **Shell** e **Bubble**. Usando vetor e arquivos.

3.2.1. Sugestão de Menu: implemente um **Menu** na aplicação para melhor interação com o usuário. Este Menu poderá conte:







3.2.2 Geração de Entradas: O código deve gerar instâncias de tamanhos: **10, 100, 1.000, 10.000, 100.000** e **1000,000** para todas as formas **(randômico, crescente e random)**, e execute o algoritmo implementado.

3.2.3 Geração de Pastas e Arquivos: O código deve gerar automaticamente as pastas e arquivos, como citado no tópico **2.3.** salvando seus respectivos valores de cada.

Na parte de arquivo, gere:

- Um **Arquivo de Entrada** contendo as instâncias geradas.
- Um **Arquivo de Saída** com as instâncias ordenadas.
- Um **Arquivo de Tempo** gasto pela ordenação.

Os arquivos tanto os de **Entrada** como o de **Saída** devem ter na primeira linha o **tamanho da instância** e o restante dos dados, um por linha, como é indicado na Figura 1. O **Arquivo de Tempo** salve apenas o tempo final gasto em segundos, ou formate em sua preferência.

Os arquivos tanto os de entrada como os de saida devem ter na primeira linha o tamanho da instância e o restante dos dados, um por linha, como é indicado na Figura 1

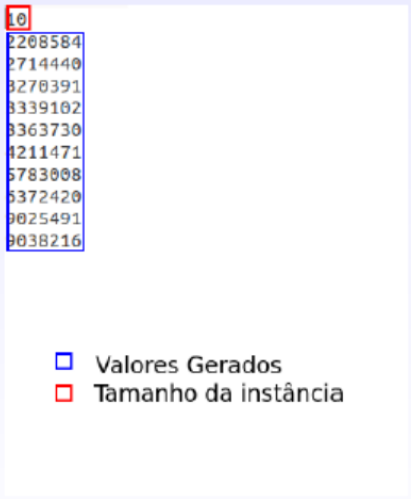


Figura: Exemplo de arquivo de entrada.

Figura 1: Exemplo de arquivo de entrada.

4. Resultado Esperado

ORIENTAÇÕES PARA ELABORAÇÃO DOS CAPÍTULOS 5

O trabalho será desenvolvido **em capítulos**, sendo que o **Capítulo 5 será dedicado ao algoritmo Merge Sort** Cada capítulo deverá conter obrigatoriamente as seguintes partes:

1. Introdução ao algoritmo

Apresente uma explicação sobre o algoritmo em questão, descrevendo seu funcionamento, sua finalidade e como o algoritmo realiza a ordenação dos elementos.

2. Teste de mesa manual

Realize um **teste de mesa manual**, passo a passo, utilizando uma entrada diferente dos exemplos trabalhados em sala de aula.

O teste deverá demonstrar claramente as alterações realizadas no vetor durante cada etapa da execução do algoritmo.

O teste de mesa deverá ser realizado manuscrito.

3. Análise e complexidade do algoritmo

Apresente a análise da complexidade do algoritmo em questão, considerando:

- melhor caso;
- caso médio;
- pior caso.

A análise deverá ser realizada conforme o procedimento apresentado em sala de aula para o algoritmo em questão, considerando o custo das instruções, o número de vezes que cada instrução é executada e os respectivos cálculos.

Esta parte deverá ser realizada manuscrita.

4. Tabelas e gráficos dos resultados

Realize os experimentos com os algoritmos e registre os tempos de execução obtidos para as diferentes instâncias.

As instâncias utilizadas deverão ser:

- 10;
- 100;
- 1.000;
- 10.000;
- 100.000;
- 1.000.000 elementos.

Os experimentos deverão considerar dados **já ordenados/Random/decrescente** .

Os resultados deverão ser apresentados por meio de tabelas e gráficos.

5. Gráficos individuais

Deverá ser elaborado **um gráfico para cada algoritmo**.

6. Tabela de comparação

Deverá ser elaborada uma **tabela de comparação dos algoritmos Insertion Sort, Selection Sort, Bubble Sort e Shellsort**.

A tabela deverá apresentar os tempos de execução obtidos para todas as instâncias utilizadas nos experimentos.

Os resultados deverão ser obtidos por meio da execução dos códigos dos algoritmos.

7. Gráfico geral

Deverá ser elaborado um **gráfico geral contendo os quatro algoritmos propostos**, permitindo a comparação de seus desempenhos para as diferentes instâncias.

O gráfico deverá considerar dados já ordenados e dados em ordem decrescente.

8. Conclusão

Deverá ser elaborada uma conclusão sobre os resultados obtidos nos experimentos.

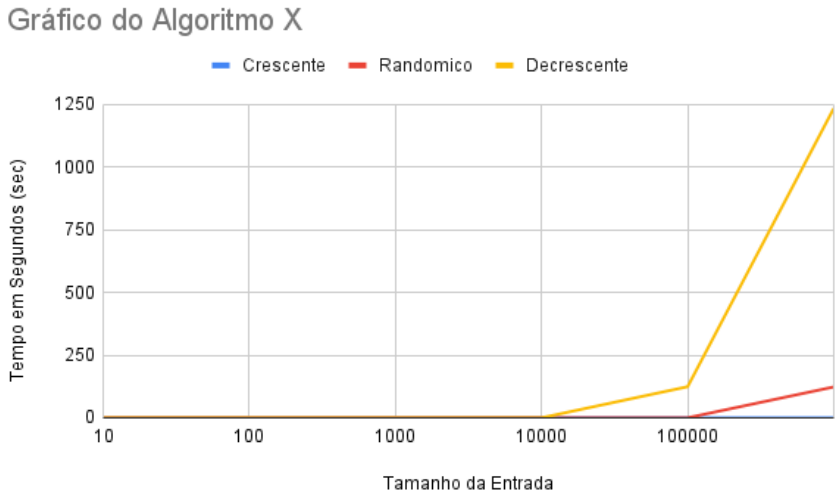


Figura 2: Exemplo de Gráfico do Algoritmo X com dados fictícios.

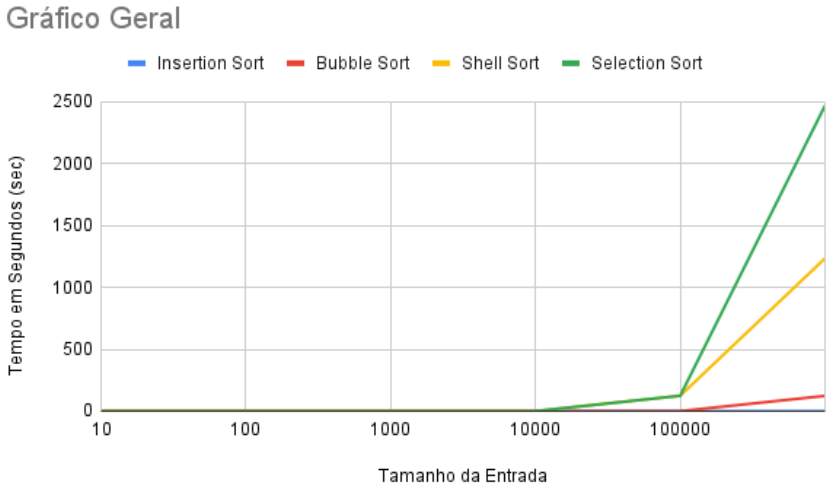


Figura 3: Exemplo de Gráfico Geral usando resultados de cada algoritmo usando dados fictícios.